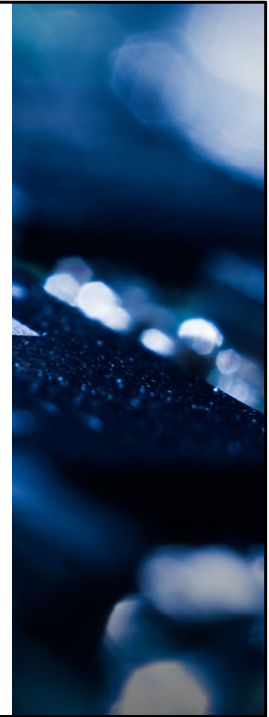




SC1006 Computer Organisation and Architecture

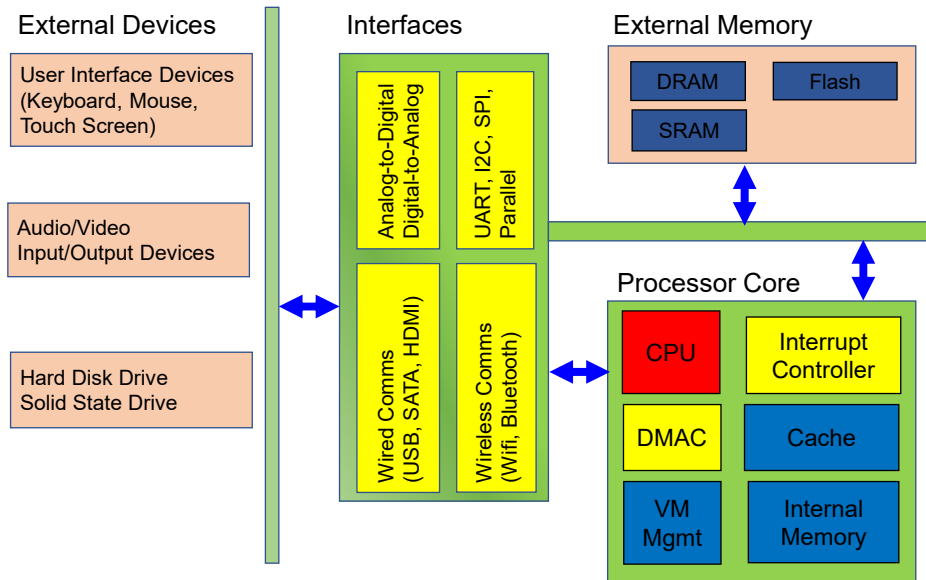
Data Transfer Mechanism

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



- Background picture
<https://pixabay.com/photos/electrician-electronics-computer-1288717/>
- **VO (for approval):**
This section is on Polling and Interrupts, which are two data transfer mechanisms commonly used in a computer system.

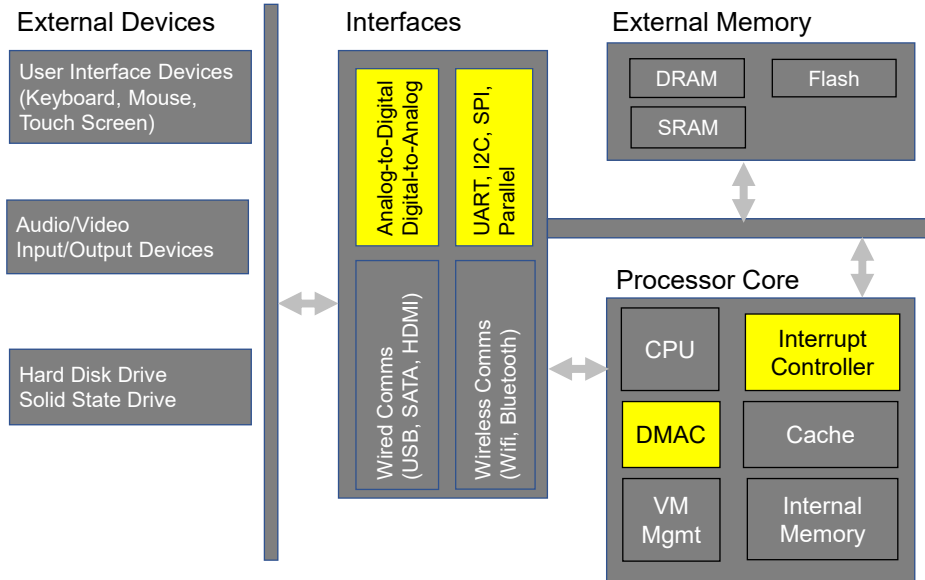
Computer System Block Diagram



VO:

Let us revisit the, computer system block diagram.

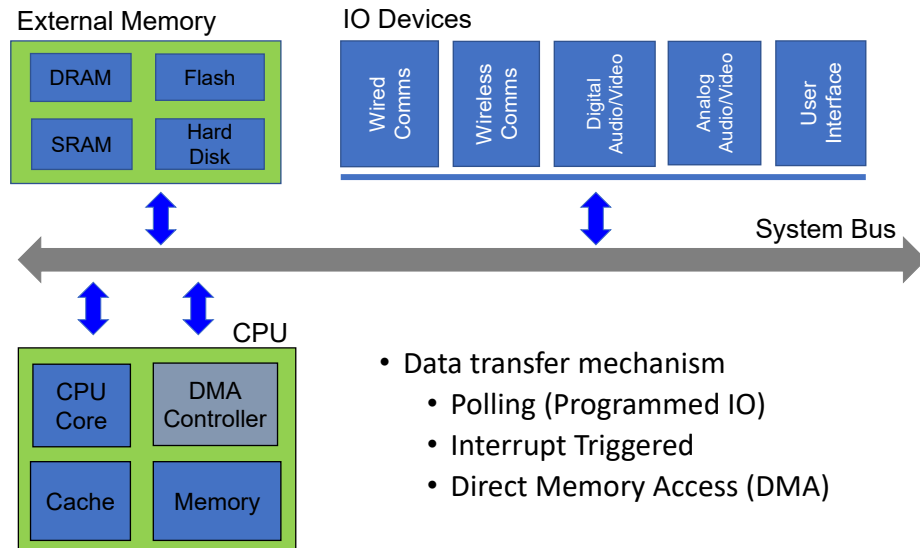
Computer System Block Diagram



VO:

In this video, we will continue, to dive deeper, into these few elements of the computer system.!!

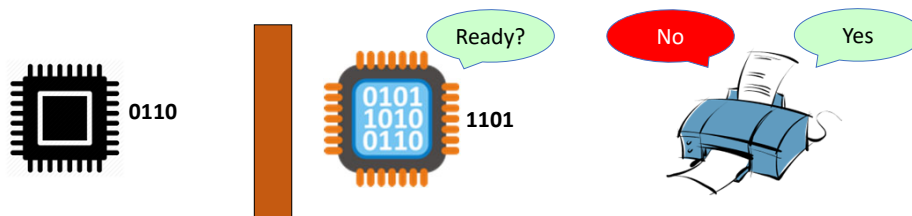
Data Transfer Mechanism



- A computer system consists of the CPU, memory and IO devices.
- These blocks are connected by a system bus, which is used to transfer data between the blocks.
- The two controllers that can initiate these transfers are the CPU core and the DMA Controller.
- The 3 transfer mechanisms that we'll touch on in this lecture are
 - Polling
 - Interrupt Triggered
 - DMA – stands for Direct Memory Access (we'll go into more details later).

Polling Technique

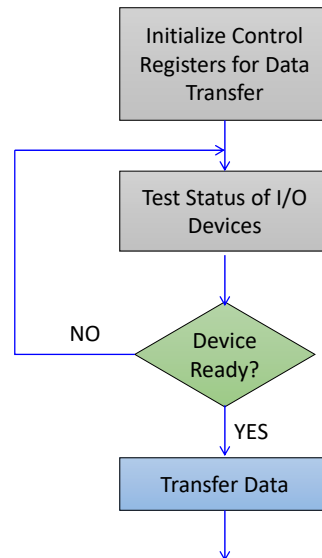
- CPU polls a certain I/O port **continuously** (using **software**) for **data** or **readiness** of the port to perform a data transaction.
 - For example, the CPU polls the printer port continuously to see if the printer is ready to accept data.
 - If it is ready, the CPU writes a data byte to the printer port. Else, it waits.
- CPU has **full control** and **dedicate 100%** of its resource in the whole data transfer process and does nothing else.



- First, Polling. Its also known as Programmed IO.
- In this method, the CPU polls a certain IO port continuously for data.
- One example is CPU polling the printer port to see if the printer is ready to accept data.
- If the printer is ready, the CPU will send the data to the printer for printing. Else, it'll wait.
- AS CPU uses 100% of its resource to do the polling, it is not able to entertain request from any other peripherals until the polling is done.
- The following animation explains how the process works.
 - Here we have a processor, printer and an external device
 - When the processor wanted to send some information to the printer, it'll first poll the printer to see if the printer is ready.
 - If the printer says 'No', the processor will continue to poll. Note that during this time, no other device will be able to get the processor's attention since it dedicate 100% of its attention on the polling.
 - Eventually, the printer will be ready and the processor can send the information over.
 - Once that is done, the process will remove the wall installed and allow other devices to communicate with it.

Polling Technique - Flowchart

- CPU performs all necessary initialization.
- CPU polls the I/O device for its readiness to perform data transfer.
- If I/O device is **not ready**, CPU continue to **wait in the loop** to check if device is ready.
- If device is **ready**, CPU makes the data transfer and **exit the loop**.



- This side shows the flow chart of the process that the CPU goes through when handling a data transfer with polling.
- CPU first performs some system initialisation
- It then polls the status of the device it wants to talk to
- If the device is not ready, it'll continue to poll the device status in a loop.
- Once the device is ready, CPU will transfer the data and exit the loop

Pros/Cons of Polling Technique

- Advantages

- Programmer has **complete control** over the entire process.
- **Easiest** method to **test** and **debug**.

- Disadvantages

- Since the CPU waits in a loop, it cannot perform any other task until data transfer is completed.
- Program execution of CPU **held up** while waiting for I/O device to get ready.
- **Inefficient use** of CPU resources.

- Advantages of using polling to transfer data is that
 - Since it is a pure software approach, the programmer has complete control over the entire process, this makes testing and debugging simple.
- However, it result in very inefficient use of the CPU resources because
 - The CPU needs to dedicate 100% of its resource to do the polling, as such it would not be able to perform any other task.
 - Specifically, program execution will be held up while CPU is wating for the devices to be ready.

Interrupts

- A **signalling mechanism** that allows internal and external peripherals to **alert** the CPU that **attention is needed**.
- Could be in the form of an external/internal **electrical pulse** or **change in internal register status**.
- Once the CPU receive the signal, known as **interrupt request**, it would then need to decide if it wanted to service the request.
- If CPU decides to service interrupt request, it will follow up with the series of procedures to handle the interrupt event.
- Interrupt mechanism is **typically used trigger the CPU to start some operation**, e.g. data transfer to memory, status registers, control registers etc.

- Interrupt mechanism is another common data transfer mechanism used and will result in a more efficient usage of CPU resources.
- Interrupt is a signal that is used to alert the CPU that its attention is needed.
- This signal can be an electrical pulse or some change in register status.
- When the CPU receives an interrupt, it can choose to service it, or not to service it.
- If the CPU decides to service an interrupt request, it will proceed to execute some pre-designed sequences. We will discuss more of that in later slides.
- In general, interrupt is typically used to trigger the CPU to start some operation, which could be some data transfer to memory, registers or other devices.

Polling vs Interrupt

- Boiling Water Analogy

Polling:

Check every few minutes



Interrupt:

Listen for the whistle



- This slide use the example of boiling water to give a simply analogy to the difference between polling and interrupt.
- If you boil water using an open pot, you will likely need to monitor the status of the water continuously or periodically to see if the water is boiled. This is similar to the polling mechanism.
- But if you boil water in a kettle with whistle, you would not need to monitor the kettle and is free to go about doing your work until you hear the whistle from the kettle. The whistle sound is similar to the interrupt request that inform the processor, which is you in this case, that your attention is needed.

Interrupt Triggered Data Transfer

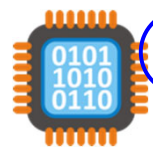
Main Routine

```
main MOV R0, #0
      MOV R2, #8

Loop ADD R1,R1,R2
      BEQ skip
      ADD R0,R0,R2

skip .....
```

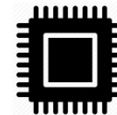
I/O Device #3
Interrupt Service
Routine



INTERRUPT REQUEST

DATA TRANSFER

I/O Device #3



interrupt latency: delay between

- the point where CPU receives the interrupt request
- To the point where it enter the ISR (Interrupt Service Routine)

- This slide will illustrate the interrupt mechanism with actual program.
- Supposed the processor is running some code in the main routine
 - After two instructions,
 - IO device #3 interrupted the CPU.
- If the CPU decides to service the interrupt,
 - the program will then branched off to a special routine known as the interrupt service routine.
 - For this case, it's a routine to transfer data between CPU and Device #3.
- After the necessary data is transferred, the control is passed back to the main routine where the code will continue where it has previously interrupted.
- One key timing parameter you need to take note is delay between
 - the point where CPU receives the interrupt request
 - To the point where it enter the ISR (Interrupt Service Routine)
- This delay is known as the interrupt latency, you will need to use this in your tutorial.

Interrupt Vector Table

Interrupt Vector Table

0	Reset ISR Addr
1	...
2	...
3	...
4	...
5	...
6	...
7	Ext INT 3 ISR Addr
8	SPI ISR Addr
9	UART ISR Addr
A	

- How does the CPU know the **location** of the corresponding interrupt service routine for each interrupts?
- The **starting address of each interrupt** is stored in a table known as the **interrupt vector table**
- Each interrupt has a **unique index to the vector table**, e.g. UART interrupt could be in index 9, so if a UART interrupt occurs, CPU will know where to branch to by checking index 9 in the vector table.
- The values and interrupt source in the table on the left are for **illustration purpose only**, different processor has different indexing for their interrupts.

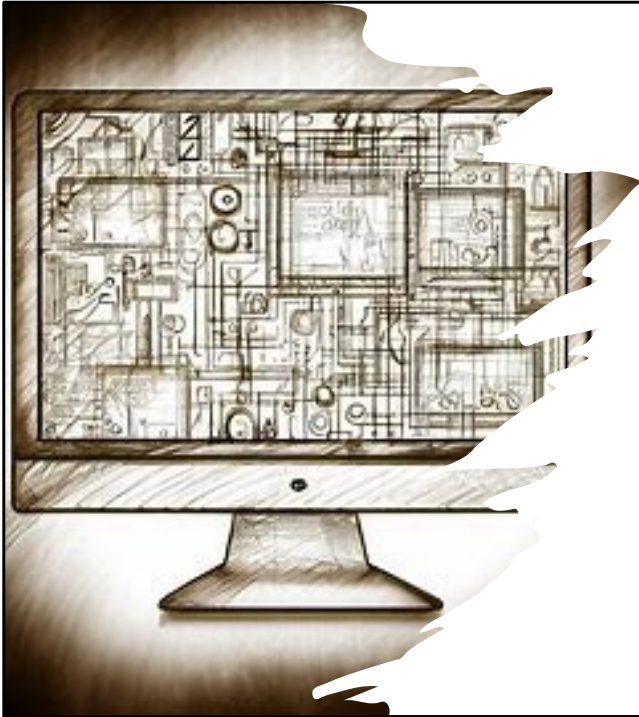
- In the previous slide, the CPU will branch off to the ISR if it decides to service the interrupt.
- But how does the CPU known where to branch to? Especially when there are many interrupt sources in the system.
- The answer is in a table known as the Interrupt Vector Table.
- This table contains the starting address of each ISR that is present in the system.
- Each interrupt has a unique location within the table so the CPU knows exactly where to look for it when searching for the starting address of the corresponding ISR.
- For example, in the table shown, UART interrupt is stored at index 9, so if a UART interrupt occur, CPU will be looking at location corresponding to index 9 for the starting address of the ISR.
- Note however, that the information in the table here is for illustration purpose only.

Interrupt Control Flow

- A signal from external/internal peripheral, or a change in status of some special registers notifies the CPU that some event has occur and ask for CPU's (immediate) attention.
- If the CPU decide to service the interrupt, it will suspend its current program temporarily.
- CPU look up the interrupt vector table to check the starting address of the interrupt service routine (ISR) for the corresponding interrupt.
- CPU would save a copy of the processor context, i.e. the current value of various registers, this is to allow the interrupted routine to continue its execution after returning from the ISR.
- CPU then proceed to execute the ISR linked to the interrupt that was triggered.
- Once the ISR is completed, CPU restore the save context, returns to the interrupted routine and continue from where it had left previously.



- This slide summarizes the interrupt control flow we discussed in the previous slides.
- You can pause the video to go through the points again.
- Note that CPU will save a copy of the processor context before branching to the ISR. This so call 'context' is basically the status of the key registers.
- This context will be restored after the ISR complete and the control is passed back to the interrupted routine, so that the interrupted routine can proceed from where it was interrupted.



Interrupt Service Routine (ISR)

- The ISR will perform some operation e.g. Data Transfer.
- ISR is typically a very short routine so as not to suspend the main program for too long.
- After servicing the ISR, CPU will return to the previous program and continue from it branched off.
- It is possible for CPU to receive multiple interrupt requests simultaneously since it typically interface to multiple devices.
- In this case, some arbitration scheme has to be designed to decide which interrupt to service first (priority, first-come-first-serve etc).

This slides further summarizes what I mentioned in the previous slides about ISR. You can pause the video to go through the points again. Some additional points that we didn't touch on in previous slides:

ISR tends to be a short routine so as not to hold the CPU for extended duration. It is possible for CPU to receive multiple interrupt requests at the same time, so CPU has to have some scheme in place to decide which interrupt to service first.

Pros/Cons of Interrupt Triggered Technique

- **Advantages**
 - **Efficient use** of CPU resources as it does not need to monitor I/O device status.
 - CPU can continue with other tasks between interrupts.
 - Allows **prioritization** and **pre-emption**.
- **Disadvantages**
 - **More hardware** interface circuitry required between I/O device and processor.
 - Program is slightly more **complex** and **difficult to debug**.



And here we have the advantages and disadvantages of interrupt triggered IO.
Advantage

It leads to a more efficient use of CPU resources

And allow processing with priority. This is One of the key enablers for high level OS as it allows

a more critical software task to have higher priority than other tasks to get the system going, e.g., to halt the current task temporarily and run the higher priority task.

Disadvantages

More hardware is needed to implement an interrupt mechanism, the control flow from interrupt request to the fetching of the ISR starting address are all handled in hardware.

Since hardware is involved, there is less visibility to the underlying operation so resultant program is more complex and more difficult to debug.